

Participants

- Customer – Ofek Goldstein [REDACTED]
- Designer – Yaron Besser [REDACTED]
- Reviewer – Nevo Alani [REDACTED]

CityPark – Municipal Parking Management System – Main Use Case

The municipal traffic authority operates city-wide parking zones and manages the complete lifecycle of parking. When a driver parks their vehicle, an employee records the session and the system calculates the applicable fee. Resident and business permit holders are registered in advance so they can park in designated zones without per-session charges. Inspectors patrol zones, issue violations for unauthorized parking, and the system tracks fine amounts and their payment status. The system also monitors zone capacity to determine real-time spot availability.

Customer Requirements

1. The system should store driver information and their registered vehicles. A single driver may have multiple vehicles.
2. The system should store parking zone details including total spot capacity, address, hourly rate, and zone type (residential, commercial, or public).
3. The system should track parking sessions with entry time, exit time, calculated total fee, and payment status.
4. The system should manage parking permits for specific vehicles in specific zones, including permit type, validity start and end dates, and the flat fee paid.
5. The system should record parking violations with the issuing inspector, violation type, fine amount, and fine payment status.

Designer Understanding of the Use Case

As the designer, I understand that the main challenges in this system are dynamic availability tracking, managing two distinct parking modes (hourly sessions versus flat-rate permits), and maintaining accountable violation records tied to specific inspectors.

I understand that a parking zone's availability cannot be stored as a simple static counter because it changes continuously as sessions start and end and as permits become valid or expire. Availability must be derived at query time.

I also understand that since one driver may own multiple vehicles, and permits and violations are attached to specific vehicles rather than to drivers, the driver and vehicle entities must be stored separately.

Discussion 1 – Organizing Driver and Vehicle Information

Discussion between Customer and Designer

Customer: We wanted to register vehicles and their owners so that employees can quickly look up who owns a vehicle that received a violation or holds a permit.

Designer: I explained that storing all driver details alongside vehicle details in a single table would create duplicated driver data whenever one driver owns more than one vehicle. Separating drivers

and vehicles into two tables prevents this redundancy and allows a driver record to be updated in one place.

Discussion between Designer and Reviewer

Designer: I proposed a dedicated Drivers table with a foreign key from Vehicles to Drivers.

Reviewer: I agreed this was the correct approach. The reviewer also noted that permits and violations could then reference specific vehicles directly, without repeating driver details on each record.

Final Decision

We agreed to maintain separate tables for Drivers and Vehicles, connected by a foreign key on the vehicle record, so that driver information is stored exactly once.

Discussion 2 – Parking Availability Tracking

Discussion between Customer and Designer

Customer: Employees needed to see how many spots are still available in a zone before approving a new parking session.

Designer: I explained that maintaining a stored available-spots counter would become inconsistent whenever sessions end, new sessions begin, or permits expire without the counter being updated at exactly the right time. Availability should be derived dynamically by comparing `total_spots` against the count of currently active sessions and valid permits in the zone.

Discussion between Designer and Reviewer

Designer: I proposed computing availability at query time from active sessions and permits rather than storing it as a field.

Reviewer: I confirmed that this approach eliminates a common class of update anomalies. The reviewer added that storing a counter would violate normalization because its value is derivable from other data already in the schema.

Final Decision

We agreed that parking availability would be calculated dynamically from active sessions and valid permits relative to zone capacity, rather than maintaining a separate available-spots field.

Discussion 3 – Session-Based vs. Permit-Based Parking

Discussion between Customer and Designer

Customer: Some drivers pay an hourly rate each time they park, while permit holders pay a flat annual or monthly fee and are not charged per session.

Designer: I explained that combining both parking modes into a single table would leave many columns null for one mode or the other, making the table difficult to query and violating normalizations. Separate `Parking_Sessions` and `Permits` tables keep each entity focused on its own attributes.

Discussion between Designer and Reviewer

Designer: I proposed `Parking_Sessions` for hourly parkers and `Permits` for flat-rate holders as two independent tables.

Reviewer: I confirmed that this separation is correct. The reviewer noted that the session fee and the permit fee are fundamentally different business concepts with different lifecycles and should not share a table.

Final Decision

We agreed to maintain separate Parking_Sessions and Permits tables to reflect the two distinct parking modes, each with its own relevant attributes.

Discussion 4 – Violation and Inspector Accountability

Discussion between Customer and Designer

Customer: When a violation is issued, we need to know which inspector issued it, what zone it occurred in, what type of violation it was, and whether the fine has been paid.

Designer: I explained that a dedicated Violations table with foreign keys to the vehicle, zone, and inspector would capture all required information cleanly. Storing fine_status directly on the violation record allows unpaid fines to be queried efficiently.

Discussion between Designer and Reviewer

Designer: I proposed a Violations table referencing Vehicles, Parking_Zones, and a separate Inspectors table.

Reviewer: I agreed with the separation. The reviewer noted that repeating inspector details such as full name and badge number on every violation record would create significant data duplication, so a separate Inspectors table with a foreign key reference was the correct design.

Final Decision

We agreed that violations would be stored in a dedicated table referencing the vehicle, zone, and inspector as foreign keys, and that fine_status would be maintained on the violation record to support payment tracking and enforcement reporting.

Final Agreed System Design

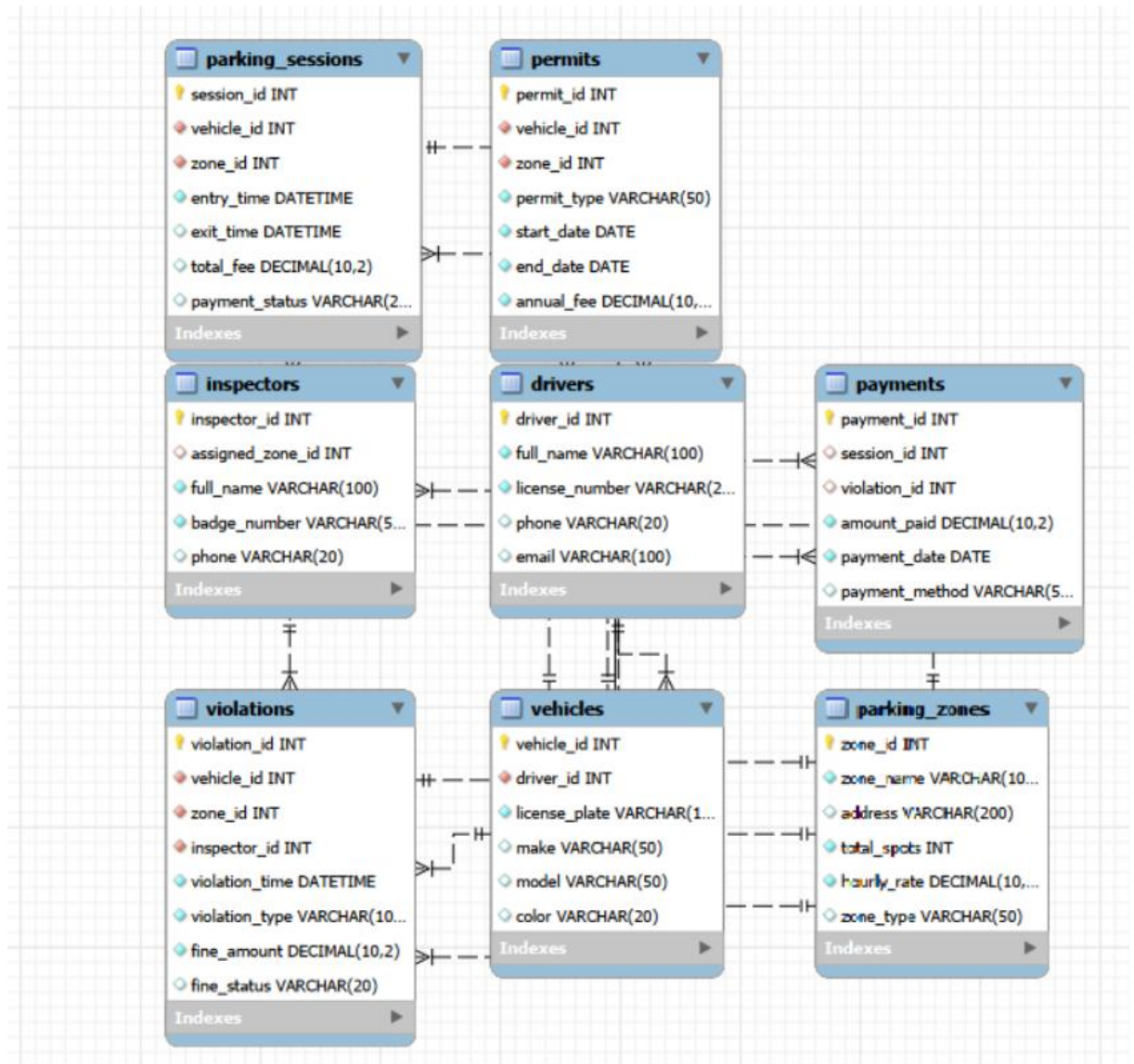
The final database design includes separate tables for drivers, vehicles, parking zones, parking sessions, permits, violations, inspectors, and payments.

The tables are connected using primary keys and foreign keys in order to maintain organized and reliable data.

The design supports:

- Dynamic parking availability calculation based on active sessions and valid permits relative to zone capacity
- Separate session-based and permit-based parking flows, each with distinct attributes
- Violation tracking with full inspector attribution, zone reference, and fine payment status

The full database structure is presented in the schema diagram.



The code:

```

1 • DROP DATABASE IF EXISTS citypark_db;
2
3 • CREATE DATABASE IF NOT EXISTS citypark_db;
4 • USE citypark_db;
5
6   -- 1. Drivers Table
7 • CREATE TABLE drivers (
8     driver_id INT PRIMARY KEY AUTO_INCREMENT,
9     full_name VARCHAR(100) NOT NULL,
10    license_number VARCHAR(20) UNIQUE NOT NULL,
11    phone VARCHAR(20),
12    email VARCHAR(100)
13  );
  
```

```

15  -- 2. Vehicles Table
16  ● ○ CREATE TABLE vehicles (
17      vehicle_id INT PRIMARY KEY AUTO_INCREMENT,
18      driver_id INT NOT NULL,
19      license_plate VARCHAR(15) UNIQUE NOT NULL,
20      make VARCHAR(50),
21      model VARCHAR(50),
22      color VARCHAR(20),
23      FOREIGN KEY (driver_id) REFERENCES drivers(driver_id)
24  );

```

```

26  -- 3. Parking Zones Table
27  ● ○ CREATE TABLE parking_zones (
28      zone_id INT PRIMARY KEY AUTO_INCREMENT,
29      zone_name VARCHAR(100) NOT NULL,
30      address VARCHAR(200),
31      total_spots INT NOT NULL,
32      hourly_rate DECIMAL(10,2) NOT NULL,
33      zone_type VARCHAR(50)
34  );

```

```

36  -- 4. Inspectors Table
37  ● ○ CREATE TABLE inspectors (
38      inspector_id INT PRIMARY KEY AUTO_INCREMENT,
39      assigned_zone_id INT,
40      full_name VARCHAR(100) NOT NULL,
41      badge_number VARCHAR(50) UNIQUE NOT NULL,
42      phone VARCHAR(20),
43      FOREIGN KEY (assigned_zone_id) REFERENCES parking_zones(zone_id)
44  );

```

```

46  -- 5. Parking Sessions Table
47  ● ○ CREATE TABLE parking_sessions (
48      session_id INT PRIMARY KEY AUTO_INCREMENT,
49      vehicle_id INT NOT NULL,
50      zone_id INT NOT NULL,
51      entry_time DATETIME NOT NULL,
52      exit_time DATETIME,
53      total_fee DECIMAL(10,2),
54      payment_status VARCHAR(20) DEFAULT 'Pending',
55      FOREIGN KEY (vehicle_id) REFERENCES vehicles(vehicle_id),
56      FOREIGN KEY (zone_id) REFERENCES parking_zones(zone_id)
57  );

```

```

59  -- 6. Permits Table
60  ● ○ CREATE TABLE permits (
61      permit_id INT PRIMARY KEY AUTO_INCREMENT,
62      vehicle_id INT NOT NULL,
63      zone_id INT NOT NULL,
64      permit_type VARCHAR(50) NOT NULL,
65      start_date DATE NOT NULL,
66      end_date DATE NOT NULL,
67      annual_fee DECIMAL(10,2) NOT NULL,
68      FOREIGN KEY (vehicle_id) REFERENCES vehicles(vehicle_id),
69      FOREIGN KEY (zone_id) REFERENCES parking_zones(zone_id)
70  );

```

```

72  -- 7. Violations Table
73  ● CREATE TABLE violations (
74      violation_id INT PRIMARY KEY AUTO_INCREMENT,
75      vehicle_id INT NOT NULL,
76      zone_id INT NOT NULL,
77      inspector_id INT NOT NULL,
78      violation_time DATETIME NOT NULL,
79      violation_type VARCHAR(100) NOT NULL,
80      fine_amount DECIMAL(10,2) NOT NULL,
81      fine_status VARCHAR(20) DEFAULT 'Unpaid',
82      FOREIGN KEY (vehicle_id) REFERENCES vehicles(vehicle_id),
83      FOREIGN KEY (zone_id) REFERENCES parking_zones(zone_id),
84      FOREIGN KEY (inspector_id) REFERENCES inspectors(inspector_id)
85  );

87  -- 8. Payments Table
88  ● CREATE TABLE payments (
89      payment_id INT PRIMARY KEY AUTO_INCREMENT,
90      session_id INT,
91      violation_id INT,
92      amount_paid DECIMAL(10,2) NOT NULL,
93      payment_date DATE NOT NULL,
94      payment_method VARCHAR(50),
95      ○ CONSTRAINT chk_payment_ref CHECK (
96          session_id IS NOT NULL OR violation_id IS NOT NULL
97      ),
98      FOREIGN KEY (session_id) REFERENCES parking_sessions(session_id),
99      FOREIGN KEY (violation_id) REFERENCES violations(violation_id)
100 );

```

Reviewer Summary

As the reviewer, I verified that the final database design fits the customer requirements and follows normalisation principles.

The main improvements included separating driver and vehicle records to handle multiple vehicles per driver without duplication, replacing a static availability counter with dynamic computation, clearly separating session-based and permit-based parking into distinct tables, and ensuring violations reference a dedicated Inspectors table rather than repeating inspector details on each fine record.

Overall, the final database design provides an organised and reliable solution for managing CityPark municipal parking operations.